



King Abdul Aziz University
Faculty of Computing & Information Technology
Computer Science Department

Mid Exam (26-10-1444 / 16-05-2023)

Course: CPCS 204 (Data Structures)

3rd Trimester (Spring 2023)

Time: 80 minutes

Student ID: _____ **Section:** _____

Student Name: _____

Instructors: Dr. Muhammad Umair	[]
Dr. Ali Khathlan	[]
Dr. Abdul Mohsin	[]
Dr. Asaad Ahmed	[]
Dr. Waseem Abdul Qayyum	[]
Dr. Abdullah Maresh	[]
Mr. Aman Ullah	[]

	Max. Marks	Obtained Marks
Q. No. 1 (MCQs):	20	
Q. No. 2 (Linked List):	15	
Q. No. 3 (Binary Search):	15	
Q. No. 4 (Recursion-Tracing):	10	
Q. No. 5 (Recursion-Coding):	15	
Q. No. 6 (Linked List-Coding):	15	
Q. No. 7 (Algorithm Analysis):	10	
Total Score	100	

Q1 Multiple Choice (4 points each)

1) For a non-empty doubly linked list with more than one node, after the execution of the following statements of while loop, what will `help_ptr` point to

```
help_ptr = head;
while( help_ptr.next != null ){
    help_ptr = help_ptr.next.prev;
}
```

- a) The last node
- b) Will result in an infinite loop
- c) head
- d) Null

2) Consider a contact list on a phone implemented as an array of max size 5000 which performs many operations. Which operation among the operations given below is the fastest?

- a) add a contact to end of the array
- b) insert contact at any point in the array
- c) search for a contact in the array
- d) delete contact from anywhere in the array

3) Which of the following conditions is necessary for the Binary Search algorithm?

- a) The number of elements in the array must always be odd in number
- b) The array must contain the value to be searched
- c) The array elements should be sorted
- d) The array must contain only integer data

4) What is the running time (in terms of n) of a Linear Search in an unsorted array of n elements?

- a) n
- b) $n \log n$
- c) $\log n$
- d) n^2

5) Which of the following statements is incorrect about recursion?

- a) Recursion is used to solve those larger problems that can be reduced to smaller sub problems of same form.
- b) A recursive method can execute infinitely.
- c) Every recursive method has exactly one base/termination case.
- d) A recursive method can, in most cases, be implemented by a loop

Q2 - Linked List (Short Questions) (5 points each)

Consider the following operations performed on an empty single linked list containing names.

```
insert_at_front("Faisal");  
insert_at_front("Fahad");  
insert_at_front("Khalid");
```

For the next two questions, assume `helpPtr` is pointing to the node containing "*Khalid*".

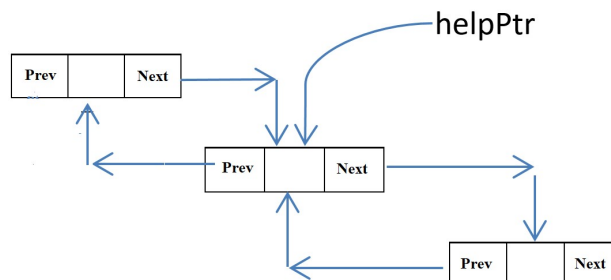
1) What will the statement (`helpPtr.next = helpPtr.next.next`) do?

- a) It makes Fahad point to Khalid.
- b) It makes Khalid point to Faisal.
- c) It makes Fahad point to Faisal.
- d) It makes Khalid point to Fahd.

2) Consider `newNode` pointing to a newly created node "*Saud*" and `newNode.next` points to *null*. What will the statement (`helpPtr.next.next.next = newNode`) do?

- a) Results in a runtime error.
- b) It adds Saud to end of list.
- c) It adds Saud before Faisal.
- d) It adds Saud before Khalid.

3) Suppose we have only 3 nodes in a doubly-linked list as shown below:



Which statements may delete the node pointed to by `helpPtr`?

- a) `helpPtr.prev.next = helpPtr.next;`
`helpPtr.next.prev = helpPtr.prev;`
- b) `helpPtr.prev.next = null;`
`helpPtr.prev = null;`
- c) `helpPtr.next.prev = null;`
`helpPtr.next = null;`
- d) None

Q3 - Binary Search (15 points)

Assume that we perform a **binary search** on an array of size n shown below to search for the number 24 (which is in the array). After execution of some steps using binary search, **assume *low* is at index 10 and *high* is at index 19**.

Index	0	1	...	9	10	11	12	13	14	15	16	17	18	19	20	...	n-1
Value	203	188	...	101	99	85	75	69	52	47	24	22	18	9	4	...	-67

What will be the values of ***low***, ***mid*** and ***high*** at each step, till you find 24?

low	Mid	high
10	14	19
15	17	19
15	15	16
16	16	16

Q4 - Recursion (Tracing) (10 points)

Trace the method given below if called as **jumble(2,15)** from another method.

```
public static int jumple(int a, int b) {  
    if (a + b >= 70)  
        return 1;  
    else  
        return 2 + jumple(a + b, b);  
}
```

You must draw all the recursive calls AND you must show the final answer that is returned.

Final Answer:

9

jumble(2,15) -> 2 + jumple(17, 15)
 2 + jumble(32, 15)
 2 + jumble(47, 15)
 2 + jumble(62, 15)
 1

Q5 - Recursion (Coding) (15 points)

Given a string as input, write a **recursive method** that will recursively compute the series given below

$$f(x, n) = \frac{1}{x^1} + \frac{1}{x^2} + \frac{1}{x^3} \dots \frac{1}{x^n}$$

Use the following method header when defined the method:

```
public static double sumOfSeries(int x, int n)
```

[Hint : The method, when called with values $x=2$ and $n=5$, should return the 0.96875. The calculation performed by the method should be:

$$\text{sumOfSeries}(2,5) = \frac{1}{2^1} + \frac{1}{2^2} + \frac{1}{2^3} + \frac{1}{2^4} + \frac{1}{2^5} = 0.96875$$

Solution

```
public static double sumOfSeries(int x, int n) {  
    if (n==1)  
        return 1.0/x;  
    else  
        return (1.0/Math.pow(x,n)) + sumOfSeries(x, n-1);  
}
```

Q6 - Linked List (Coding) (15 points)

Write a method that operates on an existing linked list of integers. Your method should delete EVERY node that has a data value ≥ 15 .

Example:

If your list has the following values (assume the first node always contains the value 0) :

0 → 51 → 4 → 2 → 9 → 12 → 7 → 81 → 6 → 13 → 24

After you run the method, your list should contain the following values:

0 → 4 → 2 → 9 → 12 → 7 → 6 → 13

The method header is given for you below (*head* points to the first node in the list):

```

public ll_node deleteOddNodes(ll_node head) {

    ll_node help_ptr = head;

    while (help_ptr.next != null) {
        if (help_ptr.next.data >= 15) {
            help_ptr.next = help_ptr.next.next;
        }
        help_ptr = help_ptr.next;
    }

}

```

Q7 - Algorithm Analysis (10 points)

Assume that an $O(n \log_2 n)$ algorithm runs for 2000 milliseconds when the input size (n) is 32. How many milliseconds will the algorithm run if the input size is doubled?

Solution:

$$T(n) = O(\log_2 n) = c * n \log_2 n \Rightarrow c = T(n) / n \log_2 n = 2000 \text{ms} / 32 \log_2(32) = 2000 / (32 * 5) = 12.5$$

$$T(n) = c * n \log_2(n) = 12.5 * (64 * \log_2 64) = 4800$$

Answer : 4800

Extra Page - If you would like any work on this page graded, please label your work clearly.